

**INTERNATIONAL CYBERSECURITY  
AND DIGITAL FORENSICS ACADEMY**

**INTRO TO CRITICAL INFRASTRUCTURE  
FOR IT LEARNERS**

**LAB 02 — PLC PROGRAMMING LANGUAGES**

**Student Edition**

Authorized ICDFA training environments only  
[icdfa.edu.ng](http://icdfa.edu.ng)

## Lab 02: PLC Programming Languages

**Course:** Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 2–3 hours

**Submission deadline:** As published in ICDFA E-Campus.

**Authorization & Safety:** Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

### Procedure

**Step 1.** Start from the clean ICDFA PLC VM snapshot and sign in to the PLC training environment.

**Step 2.** Launch OpenPLC Editor and open the Ladder Diagram example from the PLC Programming Language Examples directory.

**Step 3.** Open Res0 > instance0 (Motor\_Control). Identify the normally open Start contact, normally closed Stop contact, MotorOutput latch contact and MotorOutput coil.

**Step 4.** Open the tag/variable list. Confirm that the motor-control tags use Boolean values and record the purpose of each tag in your worksheet.

**Step 5.** Return to File > Open and load the Function Block Diagram example. Open Motor\_Control\_FBD and identify the block that maintains the motor output until a stop condition occurs.

**Step 6.** Open the Structured Text example and inspect MotorControlST. Describe how the same start/stop behavior is represented using text rather than graphical contacts and coils.

**Step 7.** Open the Instruction List example and inspect MotorControl. Record the main visual difference between this low-level textual representation and Structured Text.

**Step 8.** Open the Sequential Function Chart example and inspect MotorControl\_SFC. Identify how steps/transitions express process sequence.

**Step 9.** Complete the language comparison matrix in the lab report for all five representations.

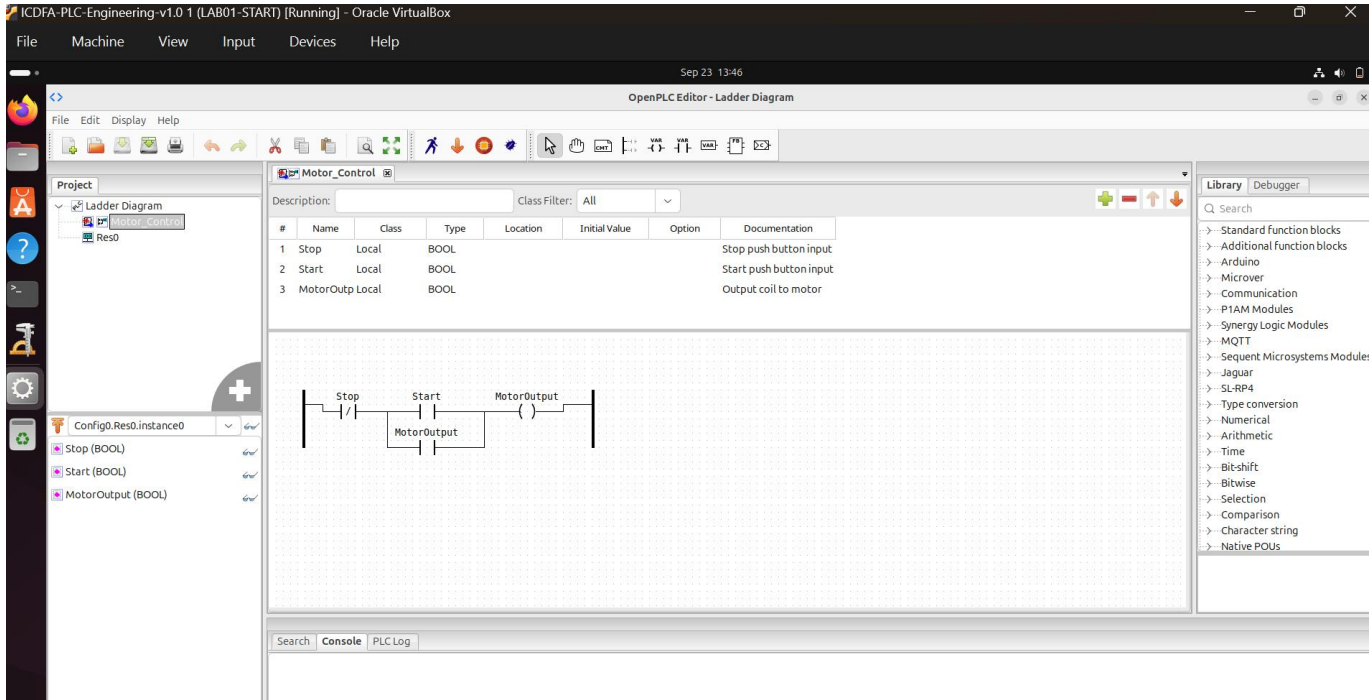
**Step 10.** Capture one evidence screenshot for each programming representation, ensuring the program name is visible.

### Required Evidence & Submission

- Five screenshots: Ladder Diagram, Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart.
- A completed comparison matrix covering representation, strengths, limitations and a suitable industrial use case.
- A short explanation showing how the same motor start/stop requirement can be represented differently while preserving the same control objective.

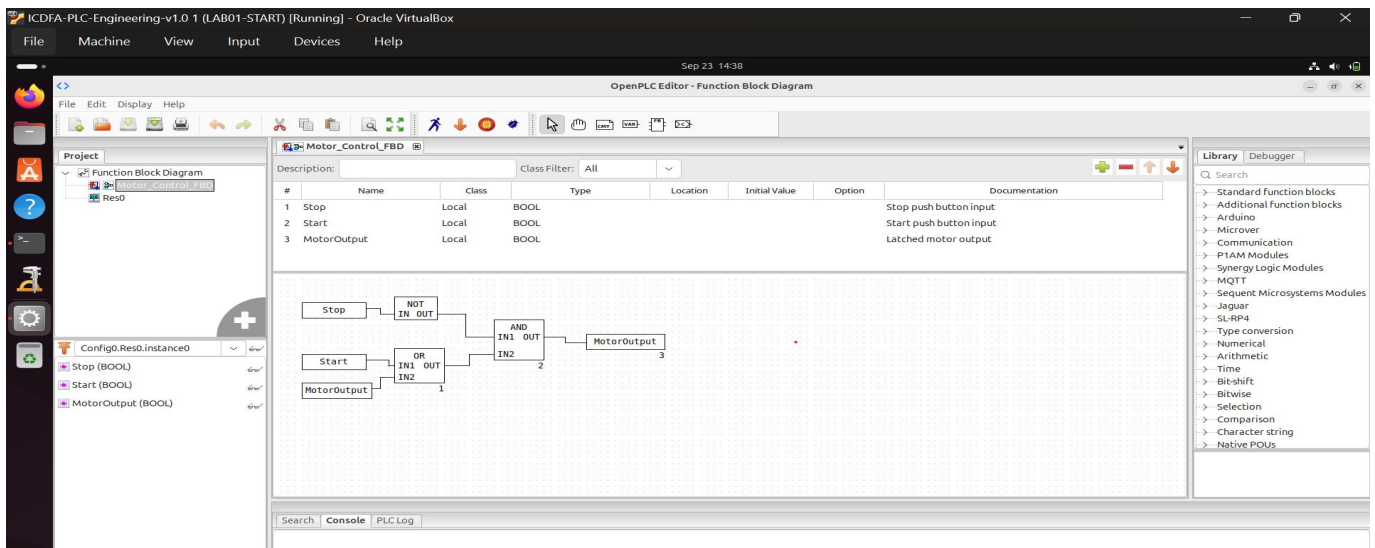
## SCREENSHOT EVIDENCES:

### 1. Ladder Diagram



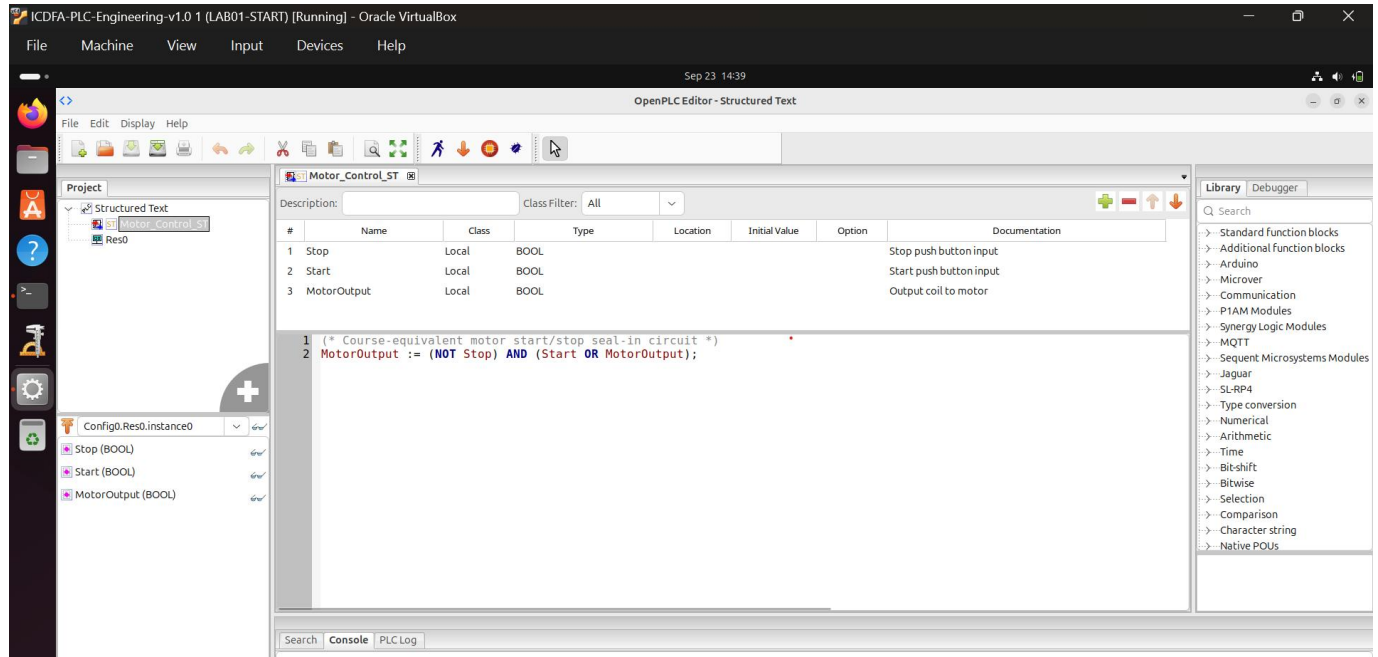
- Start: Normally Open (NO) contact.
- Stop: Normally Closed (NC) contact.
- MotorOutput (Latch): Normally Open (NO) contact connected in parallel with Start (sealing the circuit).
- MotorOutput (Coil): Output coil at the far right.

### 2. Function Block Diagram



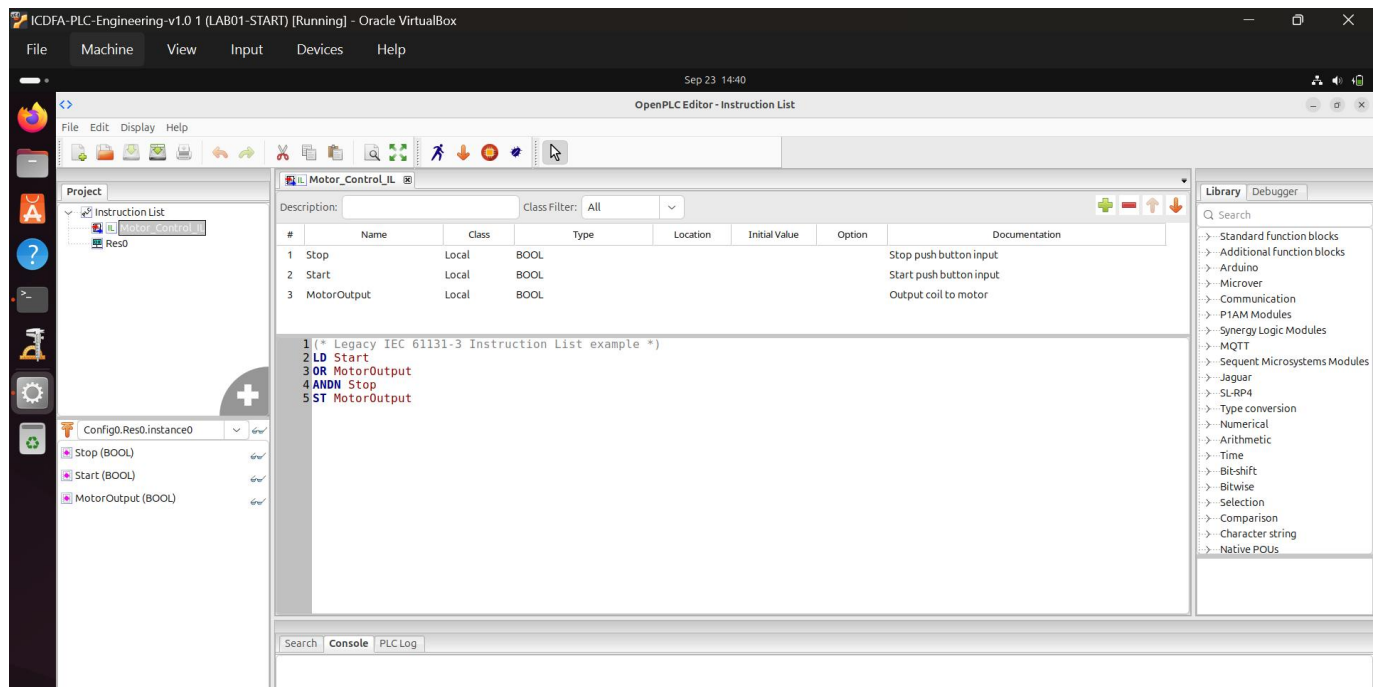
The Set pin accepts the Start condition, and the Reset pin accepts the Stop condition, latching the MotorOutput until Stop goes active.

### 3. Structured Text:



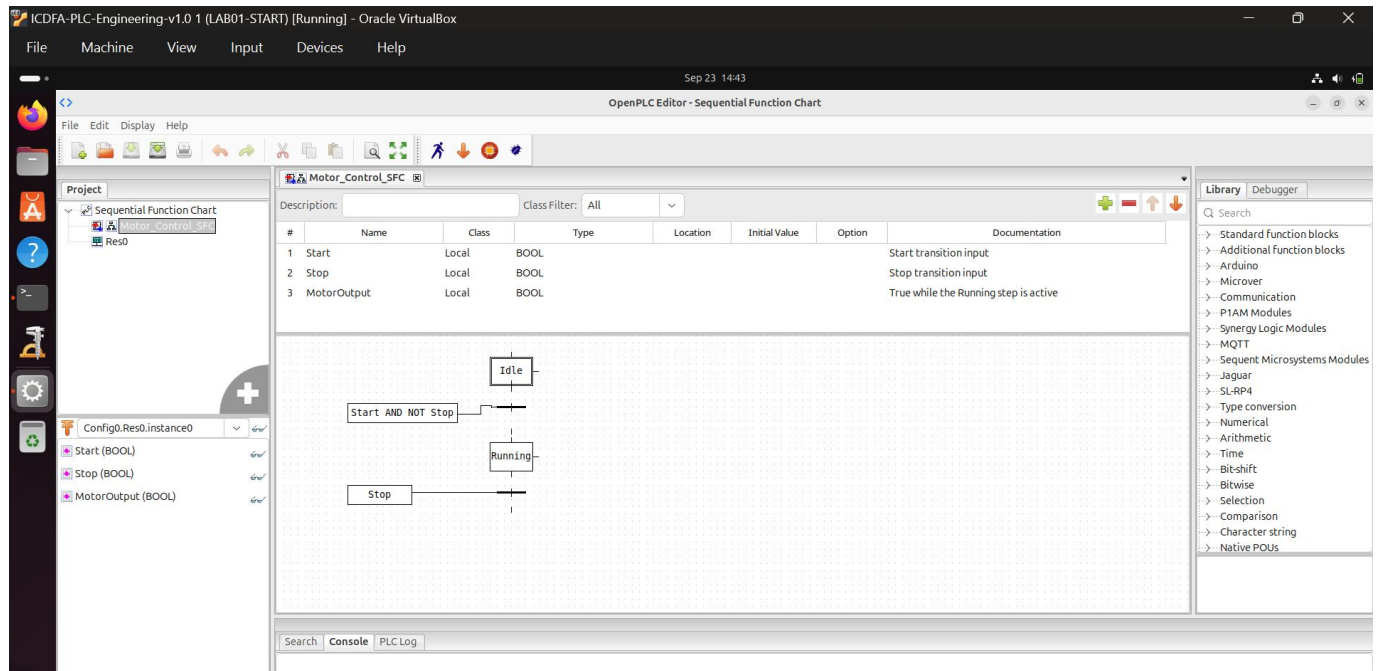
Representation Description: The motor start/stop latching logic is written as high-level conditional programming code instead of visual symbols

### 4. Instruction List



Instruction List is an assembly-like language using sequential, low-level mnemonics and an accumulator

## 5. Sequential Function Chart



The control flow is expressed as a state machine:

- **Initial Step (Step0 / Stopped):** Motor is OFF.
- **Transition 1 (Start = TRUE):** Moves execution to Step1.
- **Active Step (Step1 / Running):** Actions attached to this step set MotorOutput := TRUE.
- **Transition 2 (Stop = TRUE):** Moves execution back to Step.

### Comparison Matrix

| Language                   | Representation                                     | Strengths                                                                         | Limitations                                                           | Suitable Industrial Use Case                                          |
|----------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Ladder Diagram (LD)</b> | Graphical (looks like electrical relay schematics) | Easy to learn for electricians/technicians; simple digital logic troubleshooting. | Cumbersome for complex math, data manipulation, or looping functions. | Basic discrete motor control, relay replacement, discrete interlocks. |

|                                        |                                                                |                                                                                    |                                                                                    |                                                                          |
|----------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Function Block Diagram (FBD)</b>    | Graphical (interconnected blocks representing functions/logic) | Clear signal flow visual; excellent for process control and reusing modular logic. | Can become cluttered with large, complex programs.                                 | Analog signal processing, PID loops, safety controllers.                 |
| <b>Structured Text (ST)</b>            | Textual (High-level programming, similar to Pascal)            | Powerful for complex algorithms, data arrays, loop iterations, and math.           | Harder for field technicians to troubleshoot live on a factory floor.              | Data handling, complex calculations, recipes, string manipulation.       |
| <b>Instruction List (IL)</b>           | Textual (Low-level accumulator-based mnemonics, like Assembly) | Very fast execution speed; low memory footprint.                                   | Difficult to read, maintain, and debug; deprecated in newer IEC 61131-3 standards. | Small, legacy PLCs with tight memory/processing constraints.             |
| <b>Sequential Function Chart (SFC)</b> | Graphical (Flowchart-style with steps and transitions)         | Clearly visualizes multi-step sequential processes and state machines.             | Not suited for basic continuous or purely combinatorial logic by itself.           | Batch processing, sequential assembly lines, startup/shutdown sequences. |

## Knowledge Check

**1. Why is Ladder Diagram intuitive for technicians familiar with relay logic?**

Response: The ladder diagram is visually similar to hardwired electrical ladder schematics. It uses vertical power rails and horizontal rungs containing familiar symbols like contacts and coils. Therefore, technicians find it easier to troubleshoot.

**2. What is one advantage of Structured Text for complex calculations or conditions?**

Response: Structured text supports high-level mathematical formulas, complex conditional branches and loop structures natively. Therefore, expressing operations is easily done in a concise, readable single line of code.

**3. When might a Sequential Function Chart be more appropriate than a single ladder rung?**

Response: Sequential Function Chart (SFC) is more appropriate when managing complex, multi-step sequential processes or state-driven operations (e.g., an automated bottling line or batch chemical process). SFC breaks the control system into distinct steps and transitions, making it clear which state the machine is currently in, whereas implementing multi-step state machines across multiple ladder rungs requires extensive latching/unlatching interlocks that are difficult to trace.

**4. Why must the meaning of tags remain consistent when the representation/language changes?**

Response: Consistent tag definitions ensure that the program correctly interacts with physical I/O devices (like buttons and motors) across different languages without unpredictable behavior or safety risks.